
A Learning and Decision Framework for Collaborative Trading Agents

Master Thesis



University of
St Andrews

FOUNDED
1413

David A. Zonneveld Michel

University of St Andrews
2018

Supervisor: Alice Toniolo

Abstract

“The whole is greater than the sum of its parts.” Already expressed by Aristotle, this idea is now known as the theory of emergence. Somehow, complex systems arise from simple ones. Emergence allows complex civilizations, human beings, and nature itself to be as they are.

The same applies to Intelligent Systems. Many experiments seem to suggest that, for example, ensemble methods might be superior to single classifications/predictions. And yet, much of the effort in machine learning to date seems to be devoted to improving the predictions of single, complex entities.

For this dissertation project, a group of reinforcement-learning agents (on the order of 100) is trained to make trading decisions on three different stocks. A framework for combining their trading decisions is suggested, implemented, and evaluated. Stratified sampling strategies are suggested: Different groups of agents look at the stocks from different perspectives, using different financial abstractions of the market. Subsequently, a group of the best agents is chosen from each stratum. Different strategies for combining the different strata are tested. The end result: These 100 agents arrive at a common trading decision to maximize the rewards while reducing the uncertainty.

The results show that, in order to get better results (in this case, higher rewards), it is not only important to add multiagent techniques into machine learning projects; it is also important to have a parametrized framework where we can use validation to find the best multiagent parameters to be used.

Declaration

I declare that the material submitted for assessment is my own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated.

The main text of this project report is 13166 words long, including project specification and plan.

In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.

Contents

List of Figures	5
1 Introduction	7
2 Context Survey	11
2.1 Introduction	11
2.2 Finance and Trading	11
2.2.1 Base Indicators	12
2.2.2 Complex Technical Indicators	13
2.3 Machine Learning	16
2.3.1 Reinforcement Learning	16
2.3.2 Markov Process	16
2.3.3 Q-Learning	16
2.3.4 Hyperparameters	17
2.4 Multiagent Techniques	18
2.5 Related Works	19
3 Design	21
3.1 Framework Architecture	21
3.2 Data	24
3.3 Train/Test Data Separation	24
3.4 Data Preprocessing	24
3.4.1 Data Transformation	24
3.4.2 Scaling and Discretization	26
3.5 Reward Calculation	27
3.6 The Output	28
3.7 Single Agents	28
3.7.1 Agent State	29
3.7.2 Agent Hyperparameters	30
3.7.3 The agent's environment	30
3.7.4 Agent Training	31

3.7.5	Evaluation of State of Knowledge	32
3.8	Multiagents	33
4	Implementation	37
4.1	Libraries	37
4.2	Requirements Specification	37
4.3	Software Components	38
4.3.1	Remote Data Fetch	38
4.3.2	Local Data Fetch	40
4.3.3	Finance Component	40
4.3.4	Agent Component	40
4.3.5	Environment Component	42
4.3.6	Agent Storage Component	42
4.3.7	Agent Factory Component	42
4.3.8	Multiagent Component	43
4.3.9	Graphics Component	44
4.4	Software Testing	45
5	Evaluation and Results	47
5.1	Evaluating a Strategy	47
5.2	Adjustments Made	47
5.2.1	Stock Increase	48
5.2.2	Testing Period Length	48
5.3	Results	49
	Conclusions	53
	Acknowledgements	57
	Bibliography	59
A	User Manual	63
A.1	Configuration	63
A.2	Running	63

List of Figures

2.1	Moving Averages	13
2.2	Bollinger Bands®	14
3.1	Framework Architecture	22
3.2	Market Architecture	23
3.3	%b	25
3.4	Technical Indicators	26
3.5	Cutting Points	27
3.6	Agent Perception	27
3.7	Agent Actions	28
3.8	Unadjusted Validation Performance	32
3.9	Smoothed Validation Reward	34
3.10	Smoothed Validation Standard Deviation	35
4.1	Component Diagram	39
5.1	Evaluation Results	51

Chapter 1

Introduction

The stock market is a chaotic place. At first sight, it looks as if prices go up and down randomly. However, a closer look shows some rough patterns: If prices are slowly going up, they tend to continue going up. However, if they deviate too much, there is a probability that they will bounce back. Active trading is all about finding these patterns and trying to act accordingly.

A good part of active trading is about making trading decisions: Deciding the right moments for buying and selling (and shorting or going negative if allowed by the broker) a position. How does a trader decide when it is a good moment to go long and when it is a good moment to sell/short?

Many machine learning projects try to make these trading decisions by first trying to predict (for example, through supervised learning) the prices of the underlying asset. However, this is not an easy task. And besides, no matter how much information you have, the market will remain a stochastic place. If Bill Gates decides to sell all his Microsoft shares all of a sudden, there is not much that can be done about it; the price of Microsoft will go down with no previous warning. There will always be some unknown variables. No matter how much information we have, a prediction cannot be deterministic. Rather, it will be more like “with 5% probability the stock will increase by 2 points; with 15% it will decrease by 5 points,” etc. Possibly this could be encoded as a function, but it will be a very complicated function. Learning this function by itself might not be a problem; modern machine learning algorithms are able to learn very complicated, including probabilistic, functions.

The problem is that predicting the prices is not what traders need in the end. What traders need is to make profits. For this, they must perform trading decisions, not trading predictions. How would a trader (human or machine) make a decision from a complicated machine-learning price-prediction function? This is what this project is about: designing and evaluating a generic framework

for making these trading decisions.

Some authors suggest dropping price prediction altogether. An alternative way exists for arriving at these trading decisions without predicting the prices. Namely, reinforcement learning. Lee (2001), for example, points out that reinforcement learning might be more appropriate for modeling long-term goals and delayed rewards. With reinforcement learning, prices are not predicted. Rather, different strategies are tested, the rewards (money in this case) using different strategies are evaluated, and based on the results the strategy is adjusted.

Reinforcement learning tries to learn the actions directly. In reinforcement learning, the agent receives reward signals (either good or bad signals) from the outside world. Then, the agent tries to learn a sequence of actions (known as the policy) that maximizes not only the immediate reward, but some combination of immediate and future rewards. For doing this, the agent keeps an internal representation of the world and of the rewards of applying different actions at the different states. (Sutton & Barto 1998)

However, it is precisely this internal representation that makes reinforcement learning problematic. One problem is that the possible states of the world are usually continuous. Nevertheless, even if using a discrete representation, continuous spaces can be approximated as discrete problems, so this might not be such a big issue. A more pressing problem, however, is that for high-dimensional states it is very expensive, and often useless, to represent these states explicitly. If one chooses a coarse representation for the state, the $state \times action \rightarrow reward$ function will be inaccurate. On the other hand, if one chooses a faithful representation of the state, keeping many points to represent it, the state will become excessively large, needing a lot more storage space and training samples to cover all possible eventualities. This is a problem. (Watkins 1989, pp. 137-140)

The solution we will experiment with here is multiagent systems. According to the definition of Wooldridge (2009), multiagent systems are systems with multiple agents where the different agents have some degree of autonomy, but at the same time interact with one another for improving their decisions. In addition to dealing with the complexity issue, it might be the case that significant improvements to single-agent decisions could be achieved by using multiagent systems (Ferber 1995). However, even just being able to solve this complexity issue is already a big help. It gives us a way to solve the problem at all!

A number of existing programs for making predictions and decisions exist. However, they are usually either monolithic (the equivalent of using one agent only) or, when implemented with multiagents, either only a couple of agents are used or the agents are combined in a single way. Ideally, multiple ways of

combining different agents should be tested before using a final combination. Possibly a parametrized framework is a better solution. The goal of this project is proposing, implementing, and evaluating such a framework with real-world stock data.

The framework proposal is based on the concepts of populations of agents, where each population has a local view of the market. Then, interaction, penalization, selection, and an inter-population voting take place. The results show how using different parameter values for a multiagent framework can greatly affect the results. It suggests that blindly implementing multiagent techniques might not be enough, as very different results can be obtained by varying the parameters of the system. Therefore, a full validation-test cycle, with multiagent-specific hyperparameters, needs to exist for multiagent techniques to perform to their full potential.

This work is organized as follows: Chapter 2 covers the required background information for the project. It discusses the background for the area of finance, for the area of machine learning, and for the area of multiagents. It then mentions related works in the field. Chapter 3 then presents the framework, as well as discussing individual decisions taken for the development of the software. Chapter 4 talks about the implementation. Software requirements for developing the framework are identified. Then, the individual components of the system are discussed. Chapter 5 then discusses how the evaluation was done. It contains information on what was done for choosing one agent over another, as well as the results of the final experiments and evaluation of the framework using different parameters.

Chapter 2

Context Survey

2.1 Introduction

This chapter is organized as follows: Section 2.2 gives information on the finance and trading part. It provides an overview of the different trading tools available and used by active traders. These same tools will later be used by the agents. Section 2.3 then provides the artificial intelligence background and existing work. Section 2.4 discusses multiagent and ensemble techniques. It describes them in detail and mentions some of their applications. Finally, Section 2.5 discusses existing works in the area of stock market predictions using the techniques discussed in the first three sections.

2.2 Finance and Trading

Active traders try to make money from fluctuations in the markets. They try to predict when a stock is for example overbought/oversold, what direction that stock is going, and whether some tendency will probably change in the near future. For this, they analyze past data (such as stocks prices and volumes), compare it with present data (such as the current stock price) and with data of different intervals (such as the prices of the previous week as compared to the prices of the previous month). With this information they can make informed decisions and probabilistic predictions on where the market might be in one day, one week, or any desired period of time.

As for their sources of information, raw prices and volumes by themselves do not give much information about the market conditions. However, comparing prices and volumes of stocks over different periods of time gives a good idea of what the investors' expectations are, where the market is moving (known as

stock trends), what the resistance and support prices are, when the stocks are probably overbought/oversold, among other things. These transformed inputs are called *technical indicators* and are widely used by human traders. (Achelis 2001)

A quick note on terminology: Sometimes when the price of a stock gets too high, it splits in two, doubling the number of stocks and halving its price. These are called stock splits. Also, sometimes companies pay dividends, which are cash rewards to the investors, and, since when paying dividends to individuals money is leaving the company (moving from the company to the pockets of individual investors) naturally the price of the stocks for that company goes down by a corresponding amount. It would be much easier to make predictions if this did not happen. Therefore, some stock data providers, like Quandl, make available, in addition to the actual stock prices and stock volumes, the adjusted end of the day values, which are the prices (correspondingly, volumes) but taking into consideration these stock splits and dividends. These are called adjusted close prices (correspondingly volumes). For backward adjustments, the most common kind of adjustment used, the most recent date would be left unchanged (its actual value would be presented) and the prices/volumes of all previous days will be multiplied/divided by a number to make it appear that the prices/volumes increased continuously and that these splits and dividends never happened. The result will be smoother price/volume curves without the jumps due to stock splitting and dividend payment. (Thomas 2016)

2.2.1 Base Indicators

Moving Averages – SMA and EMA

Moving averages give averages of something (in this case, stock data), over predefined periods of time. These indicators can give an idea of what the market tendencies were and serve as the basis for other technical indicators. Moving averages are what traders call “lagging indicators”, giving an indication of what the tendencies of the markets in the past were. (Achelis 2001, pp. 26–29 and 133–138)

There are many popular moving averages. The main difference among them is how much weight is assigned to each different price. Two popular ones are the simple moving average, taking the simple average over each time interval, and the exponential moving average, giving more weight to more recent prices. (Achelis 2001, pp. 26–29 and 133–138)

These two moving averages were used for this project. As an example, the simple moving average for the DHI stock is shown in Figure 2.1.

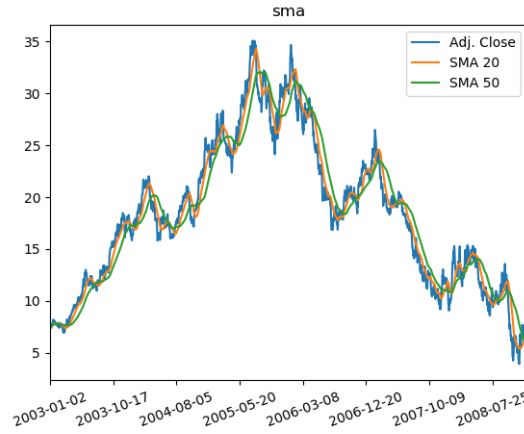


Figure 2.1: Simple Moving Average for 20 and 50 days for DHI for the years 2003-2008. It can be seen how the moving averages reflect the general tendencies of the actual price (blue line).

Bollinger Bands®

Another intermediate indicator used was Bollinger Bands®. Bollinger Bands® give the prices two standard deviations above and below the simple moving average. They indicate how high or low the prices are as compared to previous days. (Bollinger 1992)

As given by Bollinger (1992), the formulas for the Bands are:

$$Upper\ Band = \bar{X} + 2\sigma$$

$$Middle\ Band = \bar{X}$$

$$Lower\ Band = \bar{X} - 2\sigma$$

Where $\bar{X}$ is the average of the prices for a given period and σ is the standard deviation of the prices for that period.

Bollinger Bands for DHI for the years 2003-2008 can be seen in Figure 2.2.

2.2.2 Complex Technical Indicators

%b

As explained by Bollinger (1992), %b is a technical indicator derived from Bollinger Bands®. As he says, %b indicates where the adjusted close price is in relation to the upper and the lower bands. As described there, negative values indicate prices below the lower band, values between 0 and 100 prices within the bands, and values above 100 prices above the bands. That is to say,

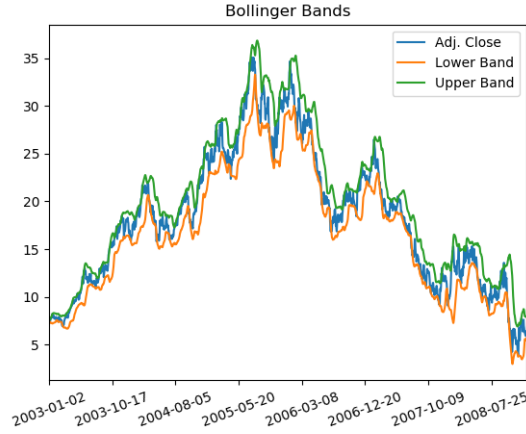


Figure 2.2: 20-days Bollinger Bands® for DHI for the years 2003-2008 superimposed on the DHI stock price.

if %b is less than 0 or more than 100 the prices are deviating more than two standard deviations from the moving average for a given period.

The formula for %b is:

$$\%b = \frac{Close - Lower\ Band}{Upper\ Band - Lower\ Band}$$

Bollinger Band Width

The second indicator derived from Bollinger Bands® is the Bollinger Band Width (BBW). As Bollinger (1992) notes, Band width expresses the width of the bands as a percentage of the moving average. According to Bollinger, the band width can be a good indicator of volatility and could predict some change of course for the stocks in the near future.

The formula for Band Width is:

$$BBW = \frac{Upper\ Band - Lower\ Band}{Middle\ Band}$$

MACD

MACD was invented by Gerald Appel (Appel 2005, p.8). As described for example in Falk & Moberg (2014, pp. 4–5), the MACD technical indicator consists of the difference between two moving averages, of a signal line calculated as the moving average of the first line, and the difference of these two lines. As he explains, MACD can be used for predicting strength, direction, and changes of trends of stocks at quite an early stage.

As explained in Falk & Moberg (2014), the formula for MACD is:

$$\begin{aligned} EMA(n)_t &= \left(\frac{2}{n+1}\right)(P_t - EMA_{t-1}) + EMA_{t-1} \\ MACD_t &= EMA(s)_t - EMA(l)_t \end{aligned}$$

where P_t is the closing value of the stock, n is the window for which EMA needs to be calculated, and s and l are a short period and a long period (that is, two additional input parameters that need to be given in addition to the window).

Then, the signal line for MACD is calculated as the EMA of the MACD.

OBV

The On-Balance Volume (OBV) indicates whether money is entering or leaving a particular stock. It is calculated by taking the cumulative stock volume data during up times and subtracting it from the cumulative stock volume data during down times. This is approximated because when a stock goes up, all the stock's trades are considered toward the up-cumulative-volumes, and similarly for the down periods. However, even if approximated it gives a good idea of how much money is going into or out of a stock and therefore whether the price will potentially increase, decrease, or change direction. (Achelis 2001)

RSI

Wilder (1978, pp. 63–70) introduced the relative strength index, RSI. As he explains, RSI tries to measure when stocks are overbought/oversold. The formula simply takes the average of the up periods for a given period, divides it by the average of the down periods during the same period, and standardizes it to the 0-100 range. As he says, by analyzing the values of the RSI, price reversals could be predicted. He suggests the limits of 30 and 70. As given by Wilder (1978, p. 65), the formula for the RSI is:

$$RSI = 100 - \frac{100}{1 + RS}$$

Where

$$RS = \frac{\text{Average of 14 day's closes UP}}{\text{Average of 14 day's closes DOWN}}$$

2.3 Machine Learning

2.3.1 Reinforcement Learning

As explained in Sutton & Barto (1998), reinforcement learning is about agents sensing the environment and learning the best policy to apply at a particular moment given a particular state of the environment; the agents will therefore have an internal representation of the state, try different actions, and observe the rewards when applying the given actions.

2.3.2 Markov Process

Markov processes were described by Dynkin (1965). A Markov Process is a process where for any time t , the probability of all future events can be fully determined by the present state, and not by past states.

2.3.3 Q-Learning

Q-Learning is a reinforcement learning technique invented by Watkins (1989). As he explains, Q-Learning can be used to learn missing information, such as the probabilities, from a Markov process.

Q-Learning goes well beyond that. Q-Learning can be used to find an optimal policy. While learning the state transitions, it learns which actions maximize the future rewards. Q-Learning can be used for iteratively trying different actions and updating the agent's knowledge on the expected reward given a certain state and a certain action. (Watkins 1989)

The Q-Function

As Watkins (1989) explains, the basic idea of Q-Learning is finding the policy that maximizes the reward for each state, action pair. One simple way of achieving this is by storing explicitly the function $Q(x, a_i)$, the reward of applying action a_i in state x , for all possible values of x .

For choosing an action in a state x , the agent then chooses an action that maximizes:

$$Q(x, a) = \rho(x, a) + \gamma \sum_y P_{xy}(a) V(y)$$

where $\rho(x, a)$ is the reward function. That is, the reward of applying action a in state x , P_{xy} is the probability that the action transforms state x into state y , and $V(y)$ is the expected return for state y .

When the agent receives a reward r when performing action a_t in state x_t we can update the Q-function with the following formula:

$$Q_{t+1} = \text{adjust}(Q_t, \langle x_t, a_t \rangle, \alpha, r_t + \gamma U_t^Q(x_{t+1}, a_t))$$

where the adjust rule could be something of the form:

$$U_{t+1}(x_t) = (1 - \alpha)U_t(x_t) + \alpha(r_t + \gamma U_t(x_{t+1}))$$

and $U^Q(x)$ can be estimated from the stored values of the Q-Function as:

$$U^Q(x) = \max_a \{Q(x, a)\}$$

That is, some linear combination of what we already knew and what we learnt. α and γ then become parameters that must be given from the outside.

2.3.4 Hyperparameters

Q-Learning has two important parameters that must be given. These are α , the learning rate, and γ , the discount rate. The learning rate specifies how fast the agent learns. That is, how fast it updates its Q-Function as new training examples come in. The discount rate then specifies how much importance the agent gives to future rewards as compared to immediate rewards. With the formula, the future rewards, according to the expected new states of the agent, are given more or less importance, depending on the value of that parameter. (Watkins 1989)

However, another important condition that must be satisfied for Q-Learning to converge to the optimal policy is given and demonstrated by Watkins & Dayan (1992):

$$\sum_{i=1}^{\infty} \alpha_{n^i(x,a)} = \infty, \sum_{i=1}^{\infty} [\alpha_{n^i(x,a)}]^2 < \infty, \forall x, a$$

That is, the learning rate must decay (otherwise the sum of its squares towards infinity will be infinite) under certain strict conditions.

The Curse of Dimensionality

The curse of dimensionality was first discussed in the context of dynamic programming. It basically means that the states become sparse when working with higher dimensions. This means that the memory required for storing all possible states grows considerably. (Bellman 1957)

Since for a base implementation Q-Learning stores the result of applying all actions to all possible states, it is clear that this is a problem for Q-Learning as well. If the size of the state space becomes too large then Q-Learning becomes impractical. This not only becomes soon intractable, but far more samples are required to cover the state space and therefore to actually have training data to cover all possible eventualities. (Watkins 1989, p. 123)

Indeed, it is easy to see that if using a matrix representation for storing the function, the size of this table grows exponentially with the number of variables to represent the states. For example, if each state is represented by three features and each feature has five possible values, possible states would be $5^3 = 125$, and the size of the matrix would be that multiplied by the number of possible actions. However, if we have five variables, each of which could have 5 possible values, the number of possible states becomes $5^5 = 3125$.

2.4 Multiagent Techniques

Agents are systems with some degree of independence capable of interacting with the world and other agents. By designing multiple independent agents, behaviors of societies can be simulated. Multiagent techniques consist of designing these systems with a series of agents acting together to achieve some common goals. Designing a multiagent system consists of designing individual systems on the one hand and designing some population strategy on the other hand. These are not always independent tasks. The design decisions and requirements for individual agents might be influenced by the design decisions and requirements for the population and vice-versa. (Wooldridge 2009)

Ensemble techniques are techniques for combining different results into one. They are widely used in the classification case, where different classifiers are independently trained and their results are then combined. Two of these techniques are majority voting and weighted voting. For majority voting, the class with the most votes is chosen. However, if it is believed that some agents might have a better performance than others (for example, by testing in some other set of data) weighted voting could be used. In weighted voting, different classifiers are given different amounts of votes, influenced by our knowledge (or perceived knowledge) of the expected performance of the individual classifiers. (Zhang & Ma 2012)

Ensemble techniques can also be used in a multiagent reinforcement-learning setting. Reinforcement learning also generates some output. Even though the output is not a single class or a single number (as is the case in classification, respectively regression, problems), but an action for a given state, the actions can still be combined in the same way: given different suggestions of

actions, the one with most votes (weighted or otherwise) can be chosen. (Sun & Peterson 1999)

2.5 Related Works

A number of Machine Learning projects and experiments for predicting the stock markets have been created. Just to give some examples, Schöneburg (1990) and Nygren (2004) run experiments to predict the price of stocks with the aid of Neural Networks. Yang et al. (2002) and Kim (2003) predict markets using support vector machines.

Combination methods such as bagging (Breiman 1996) and boosting (Kearns 1988) were successfully used for improving the efficiency and accuracy of classifiers and predictors. Another successful ensemble algorithm popular these days is Random Forests (Breiman 2001).

Some promising results exist that expand this to reinforcement learning. Two examples are Lauer & Riedmiller (2000) and Wiering & Van Hasselt (2008).

An interesting group of ensemble methods, usable in supervised and reinforcement learning alike, are voting methods. Simple Voting and Weighted Voting are two of them. Simple Voting takes a group of classifiers (respectively agents) and gives them an equal voting power. The final decision is the average of their decisions. In weighted voting we give more votes to different classifiers (respectively agents), depending on some criteria. (Breiman 2001)

Voting is by itself a multiagent method already. We do not depend on the results of one monolithic agent but on that of multiple separately trained agents. More sophisticated multiagent frameworks exist. Examples are stacked generalization (Wolpert 1992) and adaptive mixture of local experts (Jacobs et al. 1991).

An example of a successfully implemented multiagent framework in the area of the stock market can be found in Luo et al. (2002). It has been shown there how giving agents partial information of the environment and making them collaborate can give good results. The framework I suggest for this dissertation concentrates on the technical analysis part, but instead of having one agent per stock, multiple are created, separated in groups, each looking at a particular abstraction of the market, and combined using ensemble methods. Also, the framework created for this project can itself be varied: it introduces a set of framework-specific parameters (one indicating the number of agents taken from each stratum and one affecting the orderings for the selection). It will be shown that by varying the parameters of the framework, the performance can vary drastically; therefore, it seems that having such a parametrized framework

might be a good idea.

Chapter 3

Design

This chapter starts with a closer look at the framework. It then discusses different design decisions that were taken for the development of the system. Most of these design decisions are directly related to the theoretical background of Chapter 2. References to the corresponding sections will be given where appropriate.

3.1 Framework Architecture

The generic framework architecture suggested, implemented, and evaluated during this project is shown in Figure 3.1. Basically, different abstractions of the environment are made. For each abstraction a set of agents is trained to make decisions based on that abstraction. Then, the best k agents are taken from each of these sets (where k is a parameter, more on this later) by using a selection function with its own selection-parameters, as will be seen later in more detail.

In our case, the architecture turns into that shown in Figure 3.2. Basically, different agent populations observe different sets of technical indicators. With these, the different agents are looking at the market in different “ways” (just like human traders would do).

Therefore, even though the agents individually only observe one particular abstraction of the market, eventually the decisions of some of them are combined, leading to a final decision which actually takes into account all the different market abstractions.

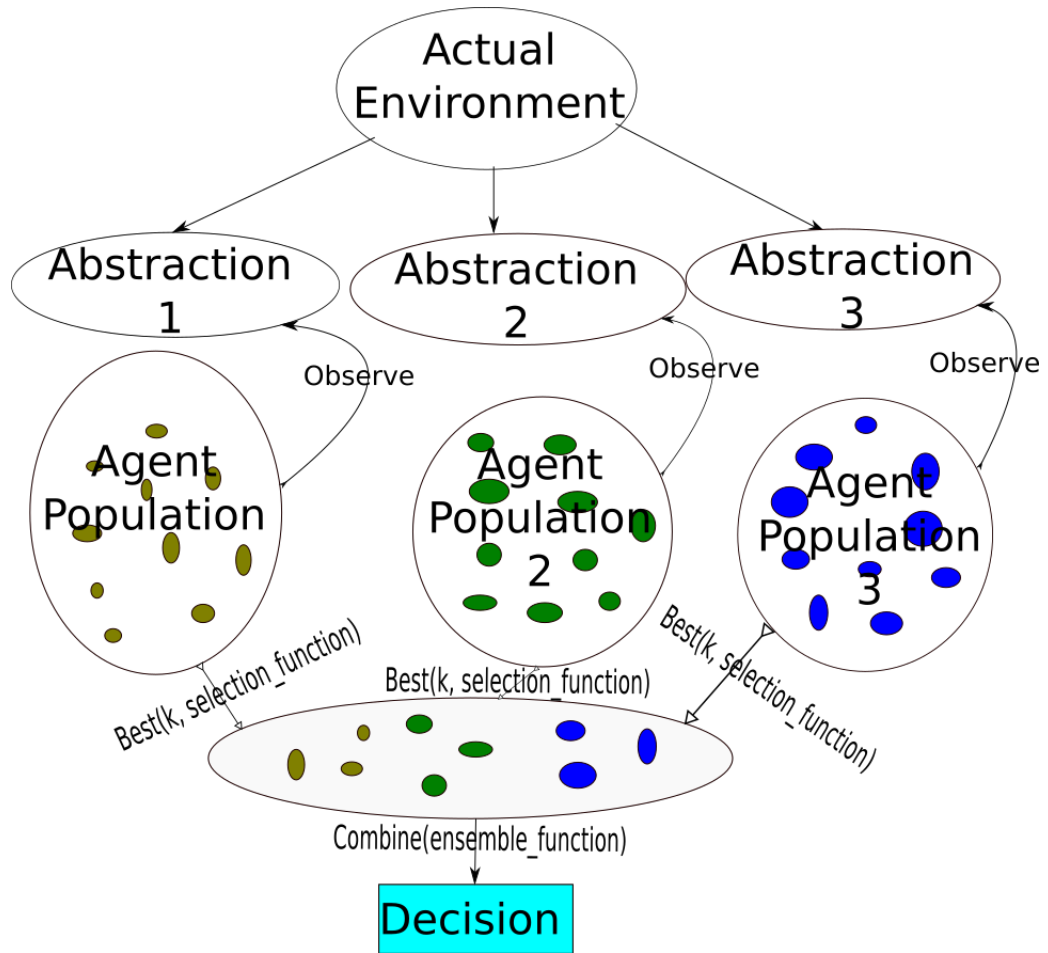


Figure 3.1: The generic multiagent architecture used. As can be seen, different agent populations observe different abstractions of the environment. Then, agents from each population are selected based on some parametrized values of a selection function (essentially creating different selection functions). The agents selected from the different populations are combined, and an ensemble algorithm is used on the resulting pool of agents.

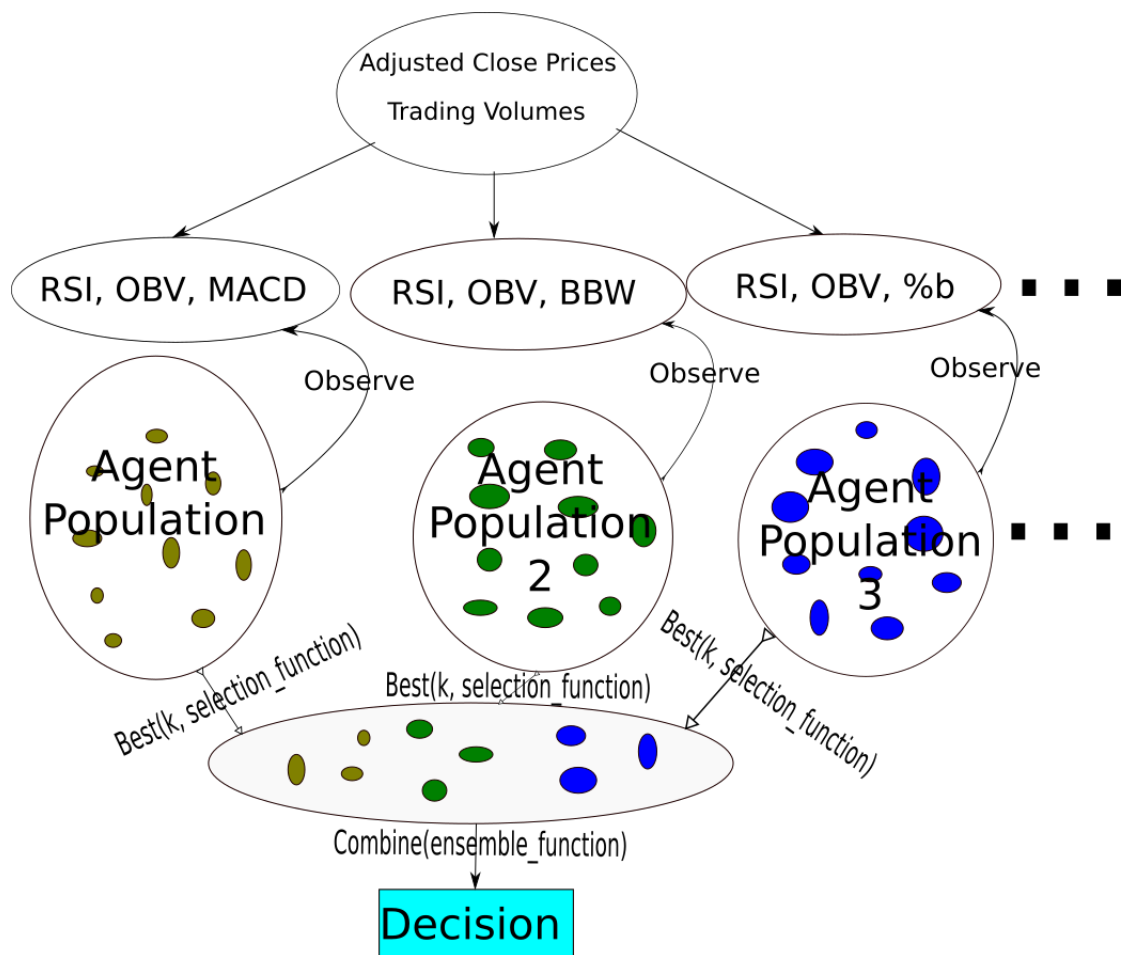


Figure 3.2: The architecture as applied to the market. Different technical indicators ensure different views of the market.

3.2 Data

The raw input data are stock trading volume data as well as historical prices at the end of each trading day adjusted to account for stock-splits and paid dividends. This data was obtained from Quandl (2018) public wiki and is accessible through their provided Python library (Quandl Developers 2006–). For this project, end-of-the-day adjusted stock prices and volumes for the years 2003 to 2017 were used. Adjusted simply means that it accounts for stock splits and dividends. The data is provided as such from Quandl.

3.3 Train/Test Data Separation

To avoid information leakage from the test set to the train set, and thus to avoid false results, the first step, before doing any preprocessing, analysis, etc., is to separate train and test data. Since we have enough data, 30% of the data was taken apart for testing. Since in the stock trading world the future contains information about the past, and to align with the goal of trying to predict future stock data with past stock data (nobody really cares about an agent that can predict the past from the future), the test data cannot be taken at random from the dataset as is usually done in Machine Learning projects. Therefore, the 30% for the test data was taken from the end.

From this point onwards, everything, except for the testing at the end, was done with the training data. For simplicity purposes the term training data will not always be explicitly mentioned, but whenever we speak about data and do not explicitly mention testing, it can be assumed that the training data is being used.

3.4 Data Preprocessing

3.4.1 Data Transformation

Raw prices and volumes by themselves do not give much information about the market conditions. However, comparing prices and volumes of stocks over different periods of time gives a good idea of what the investors' expectations are, where the market is moving (known as stock trends), what the resistance and support prices are, when the stocks are probably overbought/oversold, among other things. These transformed inputs are called technical indicators and are widely used by human traders. (Achelis 2001)

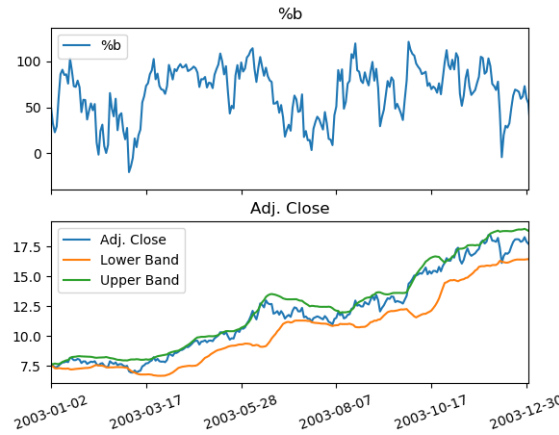


Figure 3.3: 20-days %b (above) and its corresponding Bollinger Bands[®] (below) for DHI for 2003. It can be seen, as explained in Bollinger (1992), how %b indicates, with a single number, where the stock prices are in relation to the corresponding Bands[®].

Therefore, the first step was transforming these raw inputs into more predictive ones. As described in Section 2.2, good candidates for these are technical indicators. The technical indicators used for this project were:

- RSI
- %b
- Band Width
- MACD
- OBV

These technical indicators will form the market abstractions discussed in Section 3.1. To have an idea of how this abstraction works, the values of %b for the year 2003 (Figure 3.3 top) can be compared to the actual prices of the stocks (Figure 3.3 bottom). Instead of watching the actual prices, the agents would see how well they fit into the Bollinger Bands[®].

The same applies for the other indicators. By using the formulas in Section 2.2.2, we transform the stock adjusted prices and volume information into an abstracted picture of the actual market. Each agent group will then receive a couple of these indicators, giving them a generic idea of what the market status is and how it might evolve. Even though their individual decisions will be based on a partial information of the markets, their knowledge will later on be combined through multiagent techniques.

	rsi14	obv	macd	bbw	percent_b
2003-01-23	67.93003	5914800.00000	0.01814	0.13433	78.99999
2003-01-24	68.12865	4659750.00000	0.01676	0.13320	71.87718
2003-01-27	61.15486	3527900.00000	0.00902	0.12921	44.72108
2003-01-28	64.33735	4297300.00000	0.00614	0.12570	58.15391
2003-01-29	61.55844	4297300.00000	0.00379	0.12245	57.92526
2003-01-30	50.12531	3103300.00000	-0.00299	0.12023	36.72608
2003-01-31	49.63161	3832900.00000	-0.00582	0.11748	46.08168
2003-02-03	51.60161	4450100.00000	-0.00577	0.11223	54.18132
2003-02-04	48.53937	3852650.00000	-0.00671	0.09961	46.79121
2003-02-05	52.20732	4718050.00000	-0.00557	0.08370	53.55115
2003-02-06	32.39530	3752000.00000	-0.01103	0.07765	11.58543
2003-02-07	31.16779	2968950.00000	-0.01805	0.08428	-1.53195
2003-02-10	41.01886	4321550.00000	-0.01904	0.08626	24.30112
2003-02-11	42.17846	5351950.00000	-0.01849	0.08743	31.11103
2003-02-12	34.12184	4863750.00000	-0.02320	0.09548	7.77895
2003-02-13	32.90286	3588900.00000	-0.02994	0.10831	0.26038
2003-02-14	40.51323	4689300.00000	-0.03367	0.10774	9.06171
2003-02-18	50.34153	5701100.00000	-0.02374	0.10026	65.44585
2003-02-19	46.83034	5122700.00000	-0.01951	0.09717	48.60891
2003-02-20	50.00000	4568650.00000	-0.01804	0.09374	40.08537

Figure 3.4: An example of the calculated technical indicators for the first 20 trading days. The first 14 trading days were dropped because there is enough data and most of the technical indicators require a reasonable amount of information from previous trading days to be calculated.

Since all of these indicators require information about some dates in the past to really be truthful (For example, the average of the prices of the previous 14 days is of course more trustworthy than the average of the prices of the previous 1 day), and since we have enough training data, the first 14 days were simply dropped for the actual training.

An example of what the final technical indicators would look like can be seen in Figure 3.4.

3.4.2 Scaling and Discretization

The technical indicators for a given stock are in quite different ranges. RSI goes from 0 to 100, while OBV might be in the thousands; actually, since its calculation involves volume, its range even depends on the selected stock! Other indicators, on the other hand, tend to be much, much smaller.

This is a problem in machine learning. It is well known that having different input parameters in different scale ranges will tend to favor one over the other. It is better to have the parameters in a comparable scale. Since for the way we implemented Q-Learning we furthermore need the inputs to be discrete, we combine these two.

As explained in Kotsiantis & Kanellopoulos (2006), one way to do this is by sorting the values in increasing order, selecting the desired number of bins, finding cut points so that each bin has the same number of elements, and putting data into the correct bins given the cut points. This is the approach

```

> rsi14' (112149936760) = {list} <class 'list'>: [-inf, 39.21568627451051, 51.556420233459455, 63.560524718742151, inf]
> bbw' (112149935192) = {list} <class 'list'>: [-inf, 0.12626913233156084, 0.17306950464656556, 0.23592873659325875, inf]
> percent_b' (112149924208) = {list} <class 'list'>: [-inf, 25.382499623108529, 54.553170747935312, 78.966557131410823, inf]

```

Figure 3.5: An example of the calculated cutpoints for an agent with arity four.

	rsi14	bbw	percent_b
2003-01-23	3	1	3
2003-01-24	3	1	2
2003-01-27	2	1	1
2003-01-28	3	0	2
2003-01-29	2	0	2
2003-01-30	1	0	1
2003-01-31	1	0	1
2003-02-03	2	0	1
2003-02-04	1	0	1
2003-02-05	2	0	1
2003-02-06	0	0	0
2003-02-07	0	0	0
2003-02-10	1	0	0
2003-02-11	1	0	1
2003-02-12	0	0	0
2003-02-13	0	0	0
2003-02-14	1	0	0
2003-02-18	1	0	2
2003-02-19	1	0	1
2003-02-20	1	0	1

Figure 3.6: What the technical indicators will actually look like to an agent with arity four.

that was taken for this project. The whole training data was sorted and divided into a number of categories, depending on the arity of each individual agent (arity is an agent-specific hyperparameter, as will be explained in Section 3.7.2).

An example of the calculated cutpoints for dividing the data can be seen in Figure 3.5. As can be seen, for each technical indicator the agent defines what will be the limits to fit the technical indicators into one or another category. An example of the categories that were assigned to the technical indicators values of Figure 3.4 can be seen in Figure 3.6.

Later, for testing, the test data must undergo the same transformations that the train data did. Therefore, the scaler/discretizer associated with each agent must also be saved for later.

3.5 Reward Calculation

As for the rewards of the agents, the return on investment (ROI) was used. The daily ROI was first calculated by subtracting for each trading day the value of the previous trading day and dividing the result by that same price. That is, it was calculated how much a unit investment would give in one trading day. This gives how much an agent would have won/lost if it had invested one unit in a trading session. As will be seen later, the long-term reward will be

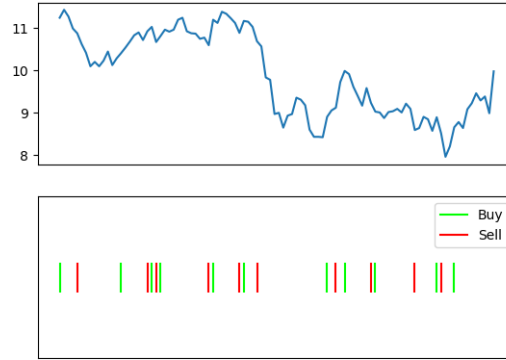


Figure 3.7: The actions that the agent decides to take for different points in time (below). For reference, the stock prices throughout the same period are also shown (above).

taken into consideration both through the learning algorithm itself, as well as through the evaluation measures.

3.6 The Output

In our case, the output of the trading problem (and as will be seen later on, the output of both individual agents and of the framework as a whole as well) are trading decisions: Decisions of when to buy and sell a stock. Part of these trading decisions is shown in Figure 3.7. Ideally, the set of trading decisions should include selling decisions soon after or before a stock starts falling and buying decisions soon after or before a stock begins to rise. Between the buy and sell decisions we do not do anything. We keep the stock (respectively, refrain from buying it) with the hope that it will keep going up (respectively, down) and our investment will keep making profits (respectively, avoid losing money).

3.7 Single Agents

The trading problem can be modeled perfectly as a reinforcement learning problem. For every point in time, each agent sees the state of the environment (actually, the abstraction of the environment as discussed before) as a combination of technical indicators. Given a group of technical indicators, the agent decides what to do next. It evaluates the results of its actions by observing

its reward and updates its knowledge of its reward for the next time it sees similar values for the technical indicators. Since values are now discretized, similar values will automatically be perceived as belonging to the same category. Therefore, the next time the agent sees similar technical indicators, it will have a better knowledge of what it is to expect when performing a given action.

For an agent that is watching an abstraction of the market with three technical indicators at a time, the input after preprocessing would be of the form shown in Figure 3.6, while the output will be of the form shown in Figure 3.7 (bottom). That is, input is a preprocessed view of the abstraction of the market, output is a set of trading decisions. These trading decisions will have an associated total reward, whose calculation will be explained in Section 3.7.5.

3.7.1 Agent State

Internally, each agent keeps a copy of its Q-function (Section 2.3.3) in the form of a matrix. The different values for the technical indicators are stored as rows, the values for the actions as columns. Each technical indicator / action pair will then store the perceived long-term rewards.

It was noted in Section 2.3.4 that if storing the function in this form the size of the state space, and therefore of the resulting matrix, grows exponentially with the number of variables. For this, we restrict each agent to a maximum of three input variables. That is, each agent observes a maximum of three technical indicators at a time. Also, the number of different states for each technical indicator after discretization (as explained above) was limited to five. With this, each agent's state representation would have a maximum of $5^3 = 125$ states. Still manageable.

Therefore, the internal state of the agent at any given time is given by:

- The money invested in the stock.
- The Q-Table, encoding the perceived reward for each action at each state. This is both the model the agent will use when determining which action to take in a given time, as well as the training status of the agent. If the agent receives new data, it “continues where it left off” by picking up the last status of its Q-Table.
- A scaler/discretizer, determining how to transform the abstract states into the agent-discretized-scaled states. That is, in which category each combination of values of the technical indicators will be put. In a way, this scaler/discretizer is the agent-specific measure of similarity for different possible values of the state.

3.7.2 Agent Hyperparameters

As discussed in Section 2.3.4, besides the state, a couple of hyperparameters must be specified for each agent.

As demonstrated in Even-Dar & Mansour (2003), with polynomial decay the convergence rate is polynomial (and not exponential, as it is with other decays) in $1/(1 - \gamma)$ (where γ is the discount factor). Therefore, polynomial learning rate decay seems like a reasonable choice. We chose the learning rate to be:

$$1/(1 + epoch)^{decay}$$

Where epoch is a full pass through the training set (more on this later) and decay is a constant.

Then, there is the discount factor. As per the learning equations of Section 2.3.3 and as explained by Watkins (1989), the discount factor defines how much weight we give to future rewards as compared to immediate rewards.

Finally, we define arity as the size of the Q-Table. Clearly, by varying the arity, the data will be discretized (as explained in Section 3.7.1) into different categories. Therefore, with a different arity, different agents will perceive different ranges of the technical indicators as being within the same or within different groups when trying to interpret the original values of the different technical indicators.

It is the different choice of these hyperparameters that distinguishes one agent from another within the same group. The hyperparameters affect their learning equation and the scaling/discretization of the states. Therefore, given the same technical indicators, different agents with different hyperparameters will learn different things.

3.7.3 The agent's environment

The environment of the agents is their abstraction of the market. In theory, every day (it could be easily adapted to be for any other interval, such as every 15 minutes or every second), each agent receives as input the subset of technical indicators that it is observing. The agent then scales/discretizes that value, as explained above, and that serves as its perception of the environment. The agent then tries some action and asks the environment what the reward would be should it have performed that action with the amount of money it has invested. The environment returns the reward of investment for the next period (in this case, the next day) multiplied by the amount invested as the calculated reward and the next state. Based on this reward, the agent updates its Q-Function using the equations described in Section 2.3.3. That is, it considers

the immediate reward plus some discounted perceived future reward according to its discount factor as given by its Q-Table.

In practice, to avoid repeated calculations and thus increase the performance, the perception of the environment by each individual agent (the scaled/discretized version) for the training set is calculated once and stored in the agent. The same goes for the reward. It is pre-calculated once for all the market's dates in a unitary way (that is, for an investment of one) and then, when requested by the agent, it suffices to return the multiplication by the agent's investment for the requested pre-calculated unitary reward for that date.

3.7.4 Agent Training

For evaluation purposes, the agent is trained with a fixed amount of data. However, the agents are designed in such a way to allow them to receive further input in the future and keep training with the new inputs.

First of all, the training data was further divided into 80% for the training set, 20% for the validation set. Again, for the same reasons described in Section 3.3, the validation could not be taken at random, but must be taken from the end. Each agent is trained for 500 iterations through the whole training data. After each iteration, the performance of the agent is measured against the validation set using the methods that will be explained in Section 3.7.5. Later on, these validation results will be used to choose the multiagent hyperparameters. This cannot be done with the testing set, as the testing set will be used in the end to evaluate the final performance and using it for choosing anything in the model (including the hyperparameters) would clearly falsify the final results.

As for the training through the training dataset for each iteration, the following procedure is applied:

1. The agent gets its perception of the scaled/discretized first state in the training set by using the procedure explained in Section 3.7.3.
2. The agent tries some valid action.
3. The agent gets the reward/next state by using the procedure explained in Section 3.7.3.
4. The agent updates its reward function (the Q-Function) considering both its immediate reward as well as its expected discounted future rewards by using the learning equations of Section 2.3.3.
5. The agent repeats this one by one for all states in the training set.

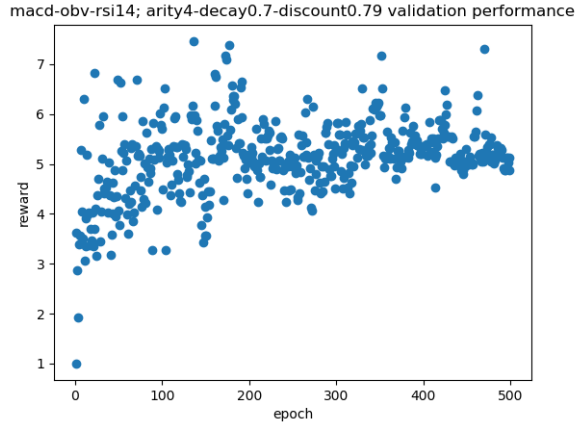


Figure 3.8: The unadjusted validation performance for one of the agents after each iteration through the whole training data set. As can be seen, the rewards from iteration to iteration vary widely. It will be seen later that, provided an appropriate penalization value is chosen, penalizing this variance when selecting which agents to take from each population might improve the predictions of the final population of agents.

3.7.5 Evaluation of State of Knowledge

As explained before, the state of knowledge of an agent is its Q-Table. Therefore, evaluating an agent amounts to evaluating the state of its Q-Table.

For evaluating the Q-Table, the following steps are followed for each of the perceived states in the validation set:

1. The agent gets its perception of the scaled/discretized first state in the validation set by using the procedure explained in Section 3.7.3.
2. The agent consults its Q-Table using the equations explained in Section 3.7.3 to find its perceived best possible action out of the set of valid actions for the current state.
3. The agent updates its money invested in the stock to reflect the application of such an action.
4. The agent asks the environment about the returns of performing its perceived best action given the amount of money currently invested in the stock.
5. The agent adds its return to 1, giving it an increase factor, when performing such an action for the given date.
6. The agent repeats this one by one, for all the test states.

7. The product of the increase factors is the increase factor of the capital for the whole test period.

The validation results of 500 iterations for one of these agents can be seen in Figure 3.8. It can be seen how the validation performance seems to increase rapidly in the first iterations and then levels off, approaching a given value. It can also be seen how varied the actual results are. This will be detrimental to the results if using this agent, as will be seen in Chapter 5, when the results of experiments penalizing agents with high variation are shown.

It must be noted that, up to this point, not all the reward increase is due to the skill of the agent. Partly, it is because the stocks actually increased in price. Corrections to account for this will be done for the final evaluation with the test set. For now, it is not necessary as this is only to compare different agents. If all the perceived rewards of all agents are scaled by the same factor, as is the case if these corrections are not done, the final selection results for the agents will be exactly the same.

3.8 Multiagents

The general idea of the multiagents framework was already given in Section 3.1. Here, more details about the multiagent parts of this framework will be provided.

The basic idea is: Given a series of different agents (for the experiments here, 100), all of them with a partial knowledge of their environment (the market), and each of them with different trading decisions, arrive at a common decision incorporating all of these agents.

As explained in Section 3.1, different groups of agents are learning on different abstractions of the market. As explained in Section 2.3.4, the state space, and thus the size of the Q-Table, grows exponentially with the number of variables used to represent the states (in this case, the technical indicators), which not only makes the program slower and the memory requirements exponentially higher, but also renders the learning useless. To avoid this problem, it was chosen that each agent group observes only 3 technical indicators at a time. There are a total of 10 different groups, each one with 3 technical indicators, none of which is exactly equal to any of the others, and each of the groups sharing two indicators with one of the other groups.

The framework selects the best k agents from each group and applies some multiagent technique over the population formed by all these agents from each group. However, this is a bit more difficult than it sounds.

The problem is how to assign an ordering from best to worst for the agents of each group. The validation data, the agent's own measure of their performance,

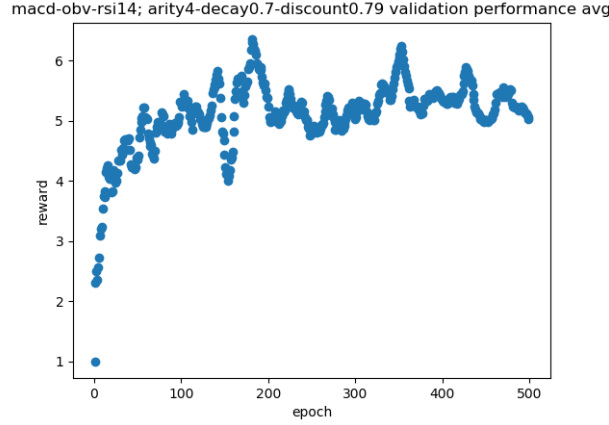


Figure 3.9: The unadjusted smoothed validation reward for one of the agents after each iteration through the whole training data set. The smoothed validation reward after iteration i is defined as the average performance of the previous 10 iterations (or, for the first 10 iterations, as many as are defined).

can be useful here. However, as was seen in Figure 3.8, the actual reward can vary by quite a lot from iteration to iteration. It would not be fair to neglect an agent just because it happened to have bad luck in the last iteration. Therefore, the first step is to smooth this reward validation curve. We define the smoothed efficiency after iteration i as the average efficiency of the previous 10 iterations. For the first 10 iterations, where this is not defined, we simply average all previous iterations. We further define the point performance as the smoothed performance at the last trained position (for our case, at iteration 500). The smoothed agent reward can be seen in Figure 3.9.

It also looks like having a big variation might be detrimental to the final prediction. An agent that always gives a reward close to five is preferable to an agent that half of the time gives a reward close to four and the other half a reward close to six.

For this, similarly to how we defined the smoothed validation performance, we define the smoothed validation standard deviation as the standard deviation of the previous 10 iterations. The smoothed standard deviation for the agent with the rewards explained above can be seen in Figure 3.10. We define the point standard deviation as the smoothed standard deviation at the last trained position (for our case, at iteration 500).

Finally, we combine these two into a performance measure, given by:

$$\text{performance metric} = \text{point performance} + \text{penalization} * \text{point standard deviation}.$$

Where point performance and point standard deviation are as defined above,

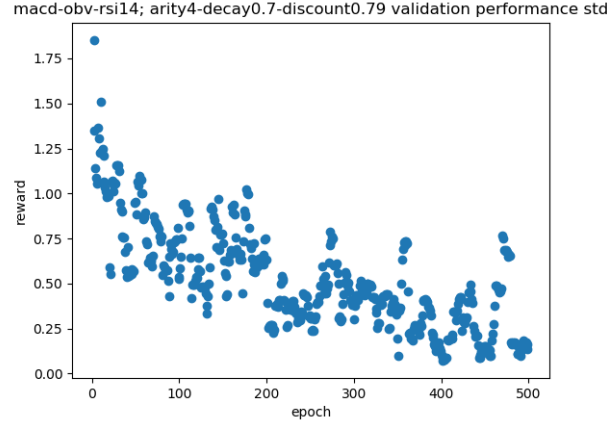


Figure 3.10: The unadjusted smoothed validation standard deviation for one of the agents after each iteration through the whole training data set. The smoothed validation standard deviation after iteration i is defined as the standard deviation (against the rewards' axis) validation reward of the previous 10 iterations (or, for the first 10 iterations, as many as are defined).

and penalization becomes a parameter specifying how much we penalize the variation of rewards within iterations for our agents.

Having the performance metric in place, we can select from each group the best k agents according to this metric. k becomes, therefore, a second parameter for selecting agents from each group.

After selecting k agents from each group, as explained above, we combine them, giving us a pool of agents from 10 different categories. The size of this pool will of course depend on the size of k .

After having this pool of agents, we apply some of the basic multiagent ensemble techniques explained in Section 2.4. Simple majority voting and weighted majority voting were tested. For simple majority voting, for each state all agents in the resulting pool were given one vote for an action. For weighted majority voting, different agents were given different amounts of votes. The amount of votes was made proportional to the performance metric which in turn, as explained above, was dependent not on the performance of the agent after the 500th iteration, but on the average performance of the last ten iterations, as well as on how much the reward varied from iteration to iteration.

The result of applying simple voting and weighted voting for the decisions of the agents for different values of k and *penalization* for three different stocks, together with the details of calculation for these results, will be discussed in Chapter 5.

Chapter 4

Implementation

In this chapter we will look at some details of the specific components, design patterns, data storage, and libraries used in the development of this project. This chapter will focus on the implementation part. The justification for each of the decisions taken here can be found in Chapter 3.

4.1 Libraries

The development was done in Python. In addition to the libraries contained in Python Standard Library 3.6.5 (Python Software Foundation 2001–), the following libraries were used:

- Numpy and Pandas (PyData Development Team 1997–, Numpy Developers 2015–), for efficient management of matrix data, including quick operations within them.
- Quandl (Quandl Developers 2006–) for accessing the Quandl stock Wiki database through their APIs.
- Scikit-learn (Sklearn Developers 2017–) for useful libraries when dealing with Machine Learning problems.
- Matplotlib and Seaborn (Hunter 2007, Waskom et al. 2017) for generating plots.

4.2 Requirements Specification

For achieving the design goals discussed in Chapter 3, the following software requirements were identified:

- A way must exist to automatically fetch any required data from the Internet.
- The fetched data might be big and used repeatedly. Therefore, a way to store and retrieve this data from the local machine must be implemented.
- Agent training might be slow. Therefore, it must be possible to store and retrieve partially trained agents.
- No two agents can be observing the same stock, with the same model abstraction, having the same machine learning hyperparameters, as these two agents would learn the exact same thing.
- Some way to preprocess the data – to convert from prices and volumes to technical indicators – must be implemented.
- It must be possible to automatically generate some graphs for all the agents.
- There must be a way to evaluate and compare the performance of different agents.
- Part of the data should be separated prior to any analysis or hyperparameter selection to have a way to impartially evaluate the final results. In other words, some machine learning test data must be separated early in the process.

4.3 Software Components

The main components used and implemented in this project for achieving the design goals described in Chapter 3 and the software requirements just identified can be seen in Figure 4.1.

4.3.1 Remote Data Fetch

The remote data fetch component is in charge of retrieving the stock data for a stock should it be unavailable in the local machine. It receives a stock, a start time, and an end time, checks if the data for that stock is available in the local machine and, if not, requests the information required for that stock for the given date-range using Quandl API. Finally, it stores the retrieved information in the shared stock data component.

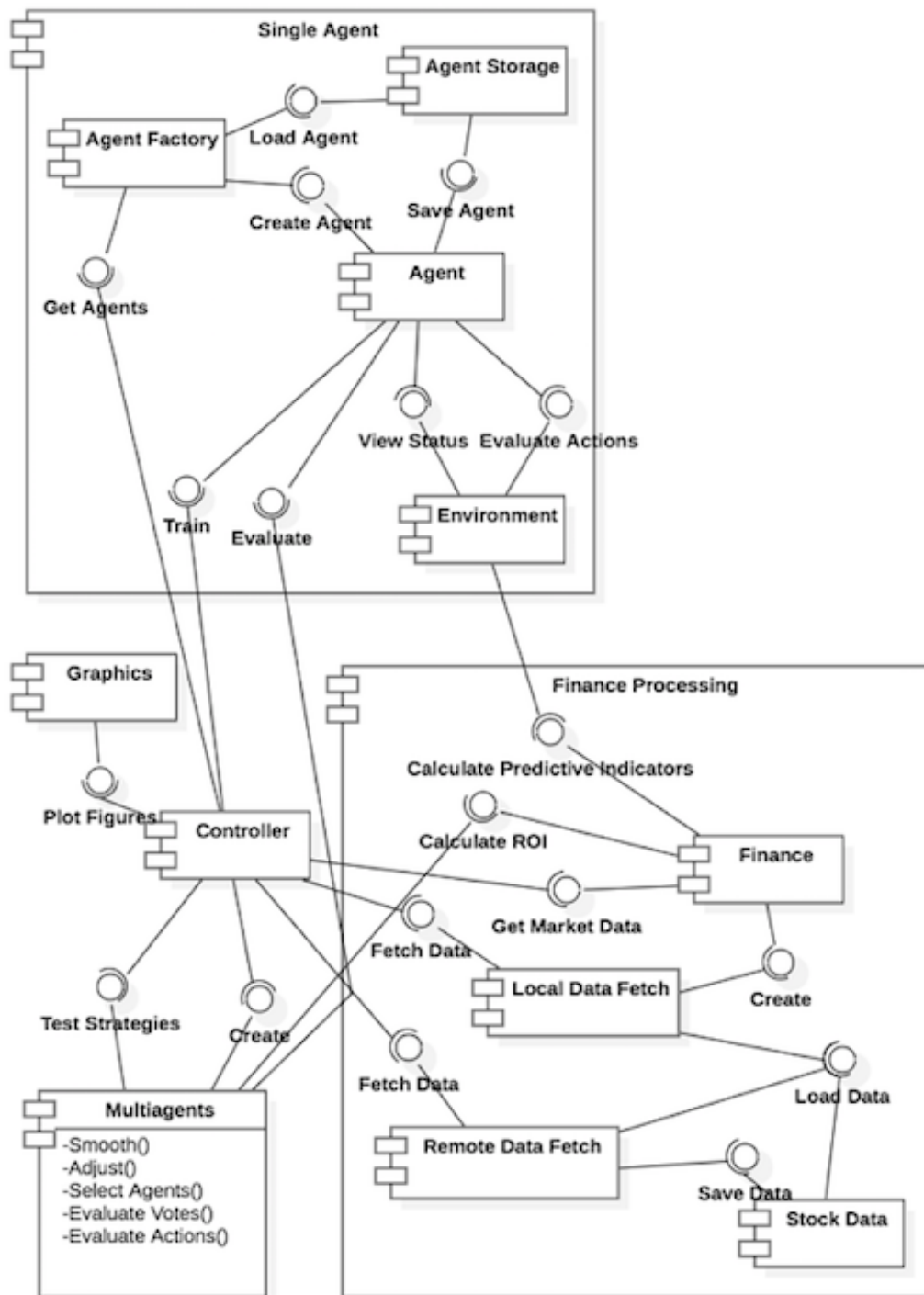


Figure 4.1: Component Diagram showing the main components implemented for this project.

4.3.2 Local Data Fetch

The local data fetcher retrieves the data stored in the shared stock data component and returns it in the form of a finance component.

4.3.3 Finance Component

The finance component encapsulates the stock price and the stock volume information for a given stock and performs all finance-related calculations with them. This includes calculating the technical indicators, calculating the unit return of investment for every specific day, generating multiple combinations of three technical indicators, and calculating the market increase for any given period.

4.3.4 Agent Component

The agent component contains all the logic for a single agent. As explained in Section 3.7.1, the agent state consists of its q-function, a scaler/discretizer, and the money invested in a stock. The q-function is stored in the form of a matrix, the scaler/discretizer as another component, but associated with every single agent (stored as a field) since this will describe how each agent uniquely interprets the market state. For easily comparing different strategies, we assume at this stage that an agent cannot short a stock below 0. Then, we initialize the agent's money left to 1 and whenever we receive a buy signal we diminish it to 0 (representing investing 100% in the stock).

The agent can at any time save its state. Indeed, every ten iterations of training it saves its state to ensure saving of the progress.

The agent also has an environment, which is the full view of the market. Every time an agent does an action it requests the environment to tell it how much it would have gained should it have invested what it has invested.

At creation time, the agent receives a bunch of states, separates them into training data (80%) and validation data (20%).

For training, the agent takes its 80% training data and uses it for training. As explained in Section 3.7.4, the agent is also able to receive a bunch of custom states and train on them, however, for testing purposes, we only use this 80% for the training.

As for the training itself, the agent simply:

- Updates its learning rate according to the current iteration using the formula explained in Section 3.7.2.

- Using this learning rate it iterates once through all the training states as follows:
 - It selects a random valid action (valid according to its money left).
 - It updates its amount of money invested in that stock.
 - It asks the environment for its corresponding reward.
 - It updates its Q-Function using this state/action/reward information.
- Every ten iterations, the agent saves its status in case there is any interruption during the training (which by the way is a very long process, so this might well be the case a couple of times).
- It evaluates the results of the iteration using the validation set and stores the result of the evaluation in a field for future reference.

For evaluation, it is calculated how much money the agents would have made with a strategy assuming the agent always invests all its money available, including/excluding any previous wins/losses. This is easier to calculate with factors. For example, if the agent increased its money by 1% we note this as a factor increase of 1.01. If it then increases the resulting money by 3%, there is an additional factor increase of 1.03. This 3% is over the already 1%-increased money, not over the original money. Therefore, the total increase of money for the agent after these two investments will be $1.01 * 1.03 = 1.0403$, or 4.03% (and not 4%). That is, an investment of 100 will be worth 104.03 after these two money increases.

Taking this into consideration, for evaluating the efficiency of our Q-Function the agent:

- Resets its state (sets its available money to 1).
- For each state in the testing set:
 - It selects the best action (as far as it knows) for the perception of this state by using its knowledge as stored in the Q-Function.
 - It asks the environment the result of applying this best perceived action.
 - It stores this as a factor ($1 + \text{absolute return}$).
 - It repeats this for all testing states, multiplying the factors together for the total factor increase during the whole testing period.

In mathematical terms, what this algorithm calculates is the following:

$$\text{Total Efficiency Factor} = \prod_{\text{days}} \text{Daily Factor Increase}$$

Where daily factor increase is

$$1 + \text{absolute return}$$

and absolute return is:

$$\begin{array}{ll} \text{market increase} & \text{if the agent had money invested.} \\ 0 & \text{otherwise.} \end{array}$$

4.3.5 Environment Component

The environment component contains the market for a given agent. It will contain the values for the given technical indicators observed by the agent at a given point in time, as well as the return of investments for different dates. Given an agent state at a given point, it returns the agent's reward as the unit return of investment (positive or negative) multiplied by the amount the agent has invested.

4.3.6 Agent Storage Component

Agent storage is a full directory tree containing the trained agents for the different stocks, for the different abstractions, for the different hyperparameter values for each abstraction. No two agents will have the same hyperparameters, observe the same abstraction, and observe the same stock. Indeed, if two agents were to have the same values for all these three, they would learn exactly the same and it would make no sense to have two agents created.

Naming conventions are used for easily identifying, saving, and retrieving a given agent from memory. Each trained (correspondingly, partially trained) agent has the stock name as the root of this directory tree, followed in the next level by the alphabetically ordered technical indicators (which as explained uniquely identify a market abstraction and therefore an agent group). Finally, each group has a series of agents on it, whose names are the alphabetically ordered technical indicators followed by their values and separated by a slash.

4.3.7 Agent Factory Component

To control all this mapping of names, and to ensure that no new agent is created if one already exists, it was found convenient to use the factory design

pattern. The agent factory receives a request to get an agent with a given set of characteristics (observing a given stock, in a given way – with a specific set of technical indicators – and with a given set of hyperparameters). The agent factory then searches if such an object already exists in the agent storage (as specified above). If it does, it returns such an agent. If it doesn't, it creates a new instance of the agent with the required parameters. From the outside we don't have access to the creation of the object itself, but only through the interface of the agent factory.

For convenience, the agent factory also provides an interface for getting n agents for each group. It simply traverses the agent structure (as above described) and for each group selects the requested amount of agents. It then returns the agents in groups (now as objects). That is, the agent factory completely abstracts to the other components the actual creation, storage, and retrieval of the respective agents.

4.3.8 Multiagent Component

The multiagent component is the one in charge of making standardizations and adjustments, smoothing the single-agent validation results for a good comparison of their validation results, combining different decisions into a final decision using different parameters and different agent sets, and testing that decision. All these processes will be discussed one by one.

First the smoothing of the performance. For quickly evaluating different agents, it is convenient to have some single measure of performance. How to evaluate how well an agent did? It was decided to do it as a combination of the reward it gets and how much this reward varies. Let us start with how much reward it gets:

As was explained in Section 3.8 and shown in Figure 3.8, for evaluating the reward after 500 iterations, it is not enough to take the reward after the 500th iteration, because of the high variability. Maybe the 499th reward was a closer representation to the actual performance. For having a better guess of the average reward after the i th iteration, smooth performance takes the reward values of the previous 10 iterations and takes the mean over them. The resulting answer will have in position i the average of positions $i - 10$ to position i (except for the first 10 iterations, which will have the average from 0 to i).

Having the smoothed rewards, the point performance method can get when required a single number indicating the average performance at position 500 by simply taking the smoothed rewards at position 500. We call this number the point average performance of each agent and will be used to compare different agents. The point variance is defined and calculated in a similar way, as the variance of the values from iteration 490 to iteration 500.

For efficiency purposes, we precalculate all this one time and save these results together with each agent. Therefore, each agent is now a three-tuple:

$$(Agent, point\ average\ reward, point\ variance)$$

With these precalculated values, it is easy to test our framework. The general algorithm for testing the framework was:

- For each value of the penalization parameter (tested from penalization = 0, which means no penalization, to penalization = 9, which is extreme penalization):
 - An ordering of the agents from each group is created according to the metric $point\ average\ reward + penalization * point\ variance$.
 - For each of the different values of k tested (tested from k = 1, meaning to take one agent per group, to k = 10, meaning to take 10 agents per group):
 - * The first k agents are taken from each group (note that they are now ordered, so the first k will give the best k according to the current metric which in turn depends on the penalization parameter).
 - * Simple majority voting / weighted voting is performed by the resulting pool of agents.

Where for simple majority voting, at each step (trading session) each agent is given one vote over the actions, all the votes for each action are added up, and the action with the most votes is taken.

Whereas for weighted voting, each agent is not given one vote but an amount proportional to their perceived efficiency. In fact, if the same factor is used for all agents the value of the factor is irrelevant, so for simplicity each agent is given an amount of votes equal to their calculated performance for the current metric with the current value of the penalization factor. As in simple majority voting, the action with the highest amount of votes for each trading session is the one that will be taken.

4.3.9 Graphics Component

The graphics component is in charge of creating all the graphics. It contains interfaces for receiving data in a variety of formats, transforming it into suitable inputs for Matplotlib and Seaborn, constructing the appropriate directories based on the base-paths stored in the configuration file and on the

naming conventions based on the values of the components, creating all the small details (such as captions, colors, and ticks, among others), and invoking the corresponding Seaborn/Matplotlib methods for creating an appropriate plot/plots. All other components are completely agnostic to the existence of Seaborn/Matplotlib. This not only makes it easy to change the graphic libraries if required, but also allows to easily implement functionalities such that the user could run the rest of the code without having Seaborn/Matplotlib installed (This is controlled by a series of flags, as will be explained in Appendix A).

4.4 Software Testing

This section will explain the procedures taken to check the correctness of individual modules and of the software system as a whole. This is not to be confused with Machine Learning Testing, which is the evaluation of the results of our Machine Learning algorithms and will be discussed in Chapter 5.

For individual components, `unittest`, which comes installed with Python, was used. Dummy values were created where necessary and the correctness of the calculations was tested. For example, for testing the calculation of the return of investment between two trading days, a finance component with prices of 100, 101, ... was created and it was confirmed that the unit return of investment after the first day was 1% by comparing the expected value (0.01) with the result of calculating ROI for the second position. Extreme cases (in this case for example, no return of investment for the first day, as it has no previous day) were also tested.

`unittest` was also used for testing the integration of different parts of the system. For example, it was tested whether, after reading and processing part of the data and requesting an agent from the agent factory with it, the resulting cutpoints had the correct dimension and were monotonically ascending for all technical indicators, whether the resulting Q-Table (As discussed, the Q-Function representation) had the correct number of dimensions, and so on.

Graphs were also quite useful for the testing of the system as a whole. For example, the shape of %b, side-by-side to that of Bollinger Bands® over the stock prices (as was shown in Figure 3.3), matches the expectations. As expected, it looks as if the only times it leaves the 0-100 range is when the prices leave the Bollinger Bands®. It also appears that when prices are close to the upper Bollinger Bands®, the value of %b is close to 100, as it should be. Of course, a little bit of trading experience and familiarity with the technical indicators is needed for this to be successful.

For the machine learning part things become more complicated. How do

we make sure that our agent is actually learning? Once again, a bit of familiarization with Machine Learning is needed. Plotting reward after each iteration, as was shown in Figure 3.8, is useful, and averaging the results for the previous ten iterations, as was shown in Figure 3.9, is especially useful here. With these curves we can see two things:

- On the average, the reward after each iteration seems to be initially increasing.
- The rewards don't seem to increase indefinitely, but seem to converge (which, as described by Watkins & Dayan (1992), is a characteristic of Q-Learning).

Chapter 5

Evaluation and Results

This chapter will explain the steps taken to evaluate the framework, as well as investigating the effect of varying the different parameters of it. It will also contain the results of the evaluation.

5.1 Evaluating a Strategy

As discussed in Section 2.4, the end result of combining a series of agents is also, similar to the single agent case, an action for each state. Therefore, testing the performance of the final decision taken by the multiagents is essentially the same as that explained in Section 3.7.5. Namely:

- Calculating the factor increase for each trading day given the amount of money invested in the stock after applying the action that day.
- Multiplying these factor increases together to obtain a total increase.

This gives us an efficiency from the start of the testing period to the end of the testing period. As explained in Section 3.3, this was taken to be 30% of the available data.

5.2 Adjustments Made

To have the final results in a standard, easy-to-compare form that is independent of actual stock movements, a couple of adjustments were made to the results as calculated above. This section will describe such adjustments.

5.2.1 Stock Increase

The result obtained so far is the total efficiency factor during the testing period. However, it would be interesting to know how much of this was because of the strategy of the agent and how much because the stock actually went up. For example, an increase of 50% is not that interesting if the stock actually increased by a factor of 2 (100%) during that same period. An equivalent way to see this is that the gains obtained are a combination of active and passive trading. To calculate the efficiency of a strategy as compared to simply buying and holding, we factor out the passive component. Therefore, the first adjustment made was dividing the calculated testing efficiency by the increase of the price of the stock during the same period:

$$\text{Strategy Efficiency Factor} = \frac{\text{Total Efficiency Factor}}{\text{Stock Market Increase}}$$

5.2.2 Testing Period Length

As it stands now, the results are for the testing period (30% of the data), which might seem a bit arbitrary when converted to dates (not even a whole amount of years). To solve this, the following equation was used to get the average increase factor per trading day:

$$\text{Average Daily Efficiency Factor} = \sqrt[\text{Total Testing Days}]{\text{Total Efficiency Factor}}$$

However, trading results are usually given on a yearly basis. The average trading year has 252 trading days (Asquith 1995). Therefore, to obtain average yearly results, the following equation was used:

$$\text{Average Yearly Efficiency Factor} = \text{Average Daily Efficiency Factor}^{252}$$

Indeed, the previous two equations can easily be combined into one:

$$\text{Average Yearly Efficiency Factor} = \text{Total Efficiency Factor}^{\frac{252}{\text{Total Testing Days}}}$$

For simplicity, all the results were obtained, calculated, and given as a factor. For percentage results, the following formula can be used:

$$\text{Percentage Increase} = (\text{Increase Factor} - 1) * 100$$

5.3 Results

The evaluation results when using different parameters of our framework can be seen in Figure 5.1. The exact meaning of the parameters was discussed in Section 3.8. Basically, different values of *agents per stratum* is what was referred to as k and corresponds to taking a different number of agents from each group (and with this, having a different number of agents in the final agent-pool). Since we have 10 groups, the number of agents in the final pool will be *agents per stratum* $\times$ 10 agents when applying the ensemble strategy (either simple majority voting or weighted voting). Then, varying *variance penalization* gives different orderings, and affects which agents are selected from each stratum based on their performance and their reward-variation from iteration to iteration.

Important observations that can be made from the results are:

- Weighted voting gives in general better results than simple majority voting, no matter how many agents we take from each group and how much we penalize variation to order them.
- Simple majority and weighted voting of all 100 (10 per group, 10 groups) agents simply by taking the best reward after the 500th iteration are the results in the lower left corner. This would most probably be the first implementation that comes to mind when speaking of voting. This does not give the best results either.
- Simple majority and weighted voting of only the best agent from each group is given by the top rows. This doesn't seem to be ideal either.
- Simply taking the agents who performed best after the 500th iteration (left column) does not seem to give the best results either. At least some penalization of agents with varying rewards seems to be necessary for optimal results.
- The best results seem to be somewhere in the middle, taking 2 to 5 agents and penalizing them (at least a bit) for the selection if their rewards vary too much.

Indeed, it seems that it might be worth, as is usually the case in machine learning, to consider converting these two parameters into hyperparameters, and validating them for obtaining their optimal values to be used for the real data (in our case, for the day-to-day trading). A word of caution here: If converting these parameters into validation hyperparameters, the testing results

will not be as good as displayed here: the displayed values are good partly because those parameter values are actually better, but also partly because of randomness. This is the typical testing-validation discussion. If using the parameters as hyperparameters the testing results given in this chapter would become the validation results because they were used for selecting the final model. A separate testing set, agnostic to the model-selection, would need to be used for testing the results of using this framework-specific-hyperparameter-tuning. Nevertheless, the variations in the testing set seem significant enough to consider actually doing this tuning prior to deployment.

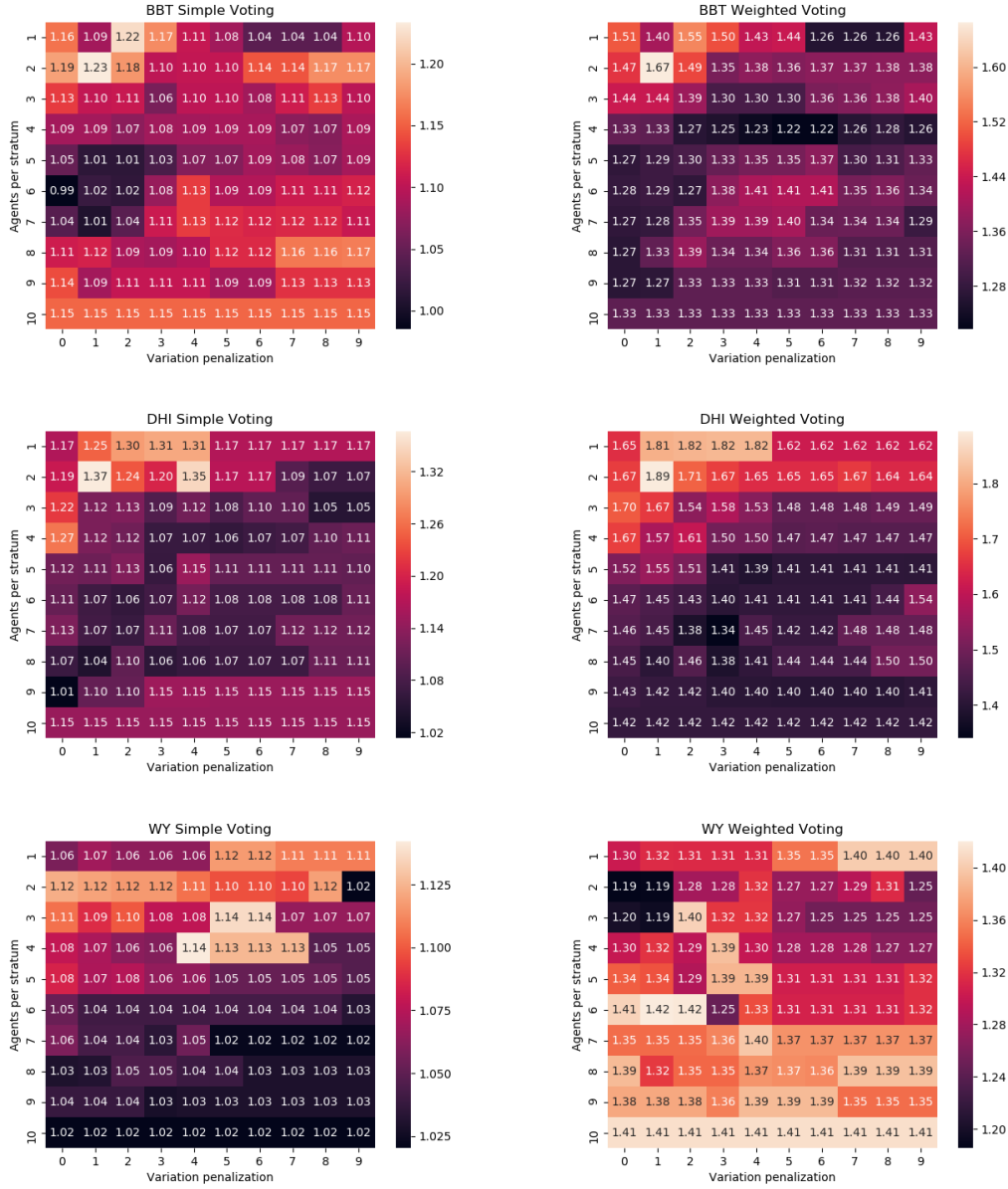


Figure 5.1: The results of running our framework with different values of the framework-specific-parameters for three different stocks. The stocks for which experiments were made are BBT, DHI, and WY. For combining the decision from the final pool of agents (after applying the filtering using the parameters), the results of both using simple majority voting (left) and weighted voting (right) are shown.

Conclusions

For this master’s dissertation, it was investigated whether a parametrized multiagent framework could give better results than simply applying ensemble methods over different trained instances. At least for the finance sector and with the limited information available, the answer is positive. The best results for our framework are not with extreme values of the parameters, which would correspond to the generally implemented ensemble techniques without any multiagent framework.

The framework was tested for three different stocks with different values of the framework-specific parameters. In none of the cases did the extreme values of the parameters give the best result: neither choosing only one nor all of the agents for each stratum. Neither ignoring an agent’s reward variation after two consecutive iterations nor penalizing it too much. The framework seems to be more successful with some intermediate values. Furthermore, these values seem to be different for different input datasets (in our case, different stocks). Therefore, for a real setting, it seems that they need to be set as machine learning hyperparameters and be subject to a machine learning validation cycle. This is typical in machine learning: we do not know in advance which values of a given set of parameters will give better results. The results seem to suggest that the same holds for combining different agents: We do not know which exact way of combining agents is better in advance. Different values must be tested for obtaining the best results.

And actually, for this project, the framework was only varied in two dimensions: number of agents chosen per stratum and penalization. Two different ensemble functions were tried. In a way, the function itself could be considered as a third parameter. For the two functions tried, given the same values for the other two parameters, weighted voting always gave better results than simple voting, no matter the values of the other parameters. This seems to indicate that we can remove simple voting as a possible value of the potential ensemble function. However, simple voting and weighted voting are not the only ensemble functions that exist. It is easy to imagine that for other ensemble functions the results might be mixed: That is, fixing the two parameters here tested

might give better or worse results depending on their values. This remains for now an open research question.

And the final ensemble function is not the only function that could be varied. For this project, the selection function was varied by varying one parameter of it, but indeed, a completely different selection function, for deciding which agents to take from each stratum, could be tested. Once again, this function could become yet another hyperparameter to enter the validation cycle when trying to optimize trading results.

One interesting advantage of this framework is the complexity reduction. Indeed, by not observing all the abstractions simultaneously, training any individual agent did not take too long. As was explained, the state space increases exponentially with the size of the feature space, so if any agent had been watching all abstractions simultaneously, at least with the current agent-implementation, this project might not have finished on time or produced any results at all!

Another advantage is the framework's immense capacity for parallelism. By using multiagents, different agents can even easily be trained independently in separate computers. As a matter of fact, for this project, because of the number of agents required, agents were trained in parallel in different clusters without the need of changing the code for adding any special support for parallelism. Later on, the trained agents were brought together for the multiagent part, but the most computationally expensive part, the training of the agents using different abstractions of the environment, was naturally parallel.

Also, the framework is quite flexible. It can easily be used with other inputs. As a proof of concept, technical indicators were fed as inputs, but other valuable inputs giving information about the market could easily be added. For example, fundamental analysis is the analysis of basic information about a company beyond its performance in the markets, investigating information on the company's tax rates, expenses, inventory, and audits, among others (Abarbanell & Bushee 1998). Opinion mining and sentiment analysis is the investigation and observation of what people feel and what people think by analyzing large amounts of data over the Internet and other sources (Pang et al. 2008). These additional sources of information could become additional inputs for some of the market abstractions. Alternatively, they could form separate abstractions by themselves.

The framework is also flexible in other aspects. The experiments were done with daily data, but could easily be switched into making minute-predictions provided minute information (known in the trading community as minute candles) is available. Indeed, any desired time-interval could be used should appropriate-length candles be provided or calculated. The framework can fur-

ther be used in alternate markets. The experiments were done with stocks, but the same algorithms could be used to predict trading decisions for gold, oil, derivatives, cryptocurrencies, or any other financial asset; once again, provided the appropriate candles are available or easily obtainable/calculated.

Indeed, the framework could even be used outside the finance world, provided a good way to obtain abstractions of the environment is available. Through technical indicators, these abstractions become natural for finance, but other areas might benefit as well from using this parametrized framework with its reduced complexity should good alternate partial views of the environment be available.

Then, the agent architecture itself could be varied as well. For example, the agents could have a shared pool of resources (money). Then, if one agent uses part of this money for buying a stock, another agent will not have it available anymore. Given this additional constraint, the agent strategy will necessarily change. The reward obtained will then not only depend on the agent's decisions but on the decisions of the group as a whole. Implementing such changes remains a future research project.

There is plenty of work to be done on the multiagent theory. Some researchers do use ensemble methods, which is a good start, but as used today, those are like the extremes of the suggested framework (or maybe even more extreme, as usually not even different populations are used). The results show interesting performance deviations when applying the same agent-combination techniques by simply varying a couple of multiagent-framework-specific parameters. Possibly more machine learning resources should be devoted to investigating different ways to better combine multiple agents, and which multiagent-specific-hyperparameters could be used and tuned to obtain better results.

Acknowledgements



With the support of the Dependable Software Systems (DESEM) Programme of the European Union.

Special thanks to the European Commission, which, through the EACEA, provides quality international opportunities for student exchanges and, in my case, for a fully funded master's programme.

Special thanks to Alice, who, as well as being my supervisor, was always available for discussing and exchanging ideas. Her patience and her knowledge of multiagent systems were extremely useful in the development of this framework.

Bibliography

- Abarbanell, J. S. & Bushee, B. J. (1998), ‘Abnormal returns to a fundamental analysis strategy’, *Accounting Review* pp. 19–45.
- Achelis, S. B. (2001), *Technical Analysis from A to Z*, McGraw Hill New York.
- Appel, G. (2005), *Technical Analysis: Power Tools for Active Investors*, first edn, FT Press.
- Asquith, P. (1995), ‘Convertible bonds are not called late’, *The Journal of Finance* **50**(4), 1275–1289.
- Bellman, R. (1957), ‘Dynamic programming, princeton, nj: Princeton univ’.
- Bollinger, J. (1992), ‘Using bollinger bands’, *Stocks & Commodities* **10**(2), 47–51.
- Breiman, L. (1996), ‘Bagging predictors’, *Machine Learning* **24**(2), 123–140.
URL: <https://doi.org/10.1007/BF00058655>
- Breiman, L. (2001), ‘Random forests’, *Machine learning* **45**(1), 5–32.
- Dynkin, E. B. (1965), Markov processes, in ‘Markov Processes’, Springer, pp. 77–104.
- Even-Dar, E. & Mansour, Y. (2003), ‘Learning rates for q-learning’, *Journal of Machine Learning Research* **5**(Dec), 1–25.
- Falk, A. & Moberg, J. (2014), ‘Algorithmic trading using macd signals’.
- Ferber, J. (1995), *Les systèmes multi-agents: vers une intelligence collective*, InterEditions.
- Hunter, J. D. (2007), ‘Matplotlib: A 2d graphics environment’, *Computing In Science & Engineering* **9**(3), 90–95.

- Jacobs, R. A., Jordan, M. I., Nowlan, S. J. & Hinton, G. E. (1991), ‘Adaptive mixtures of local experts’, *Neural computation* **3**(1), 79–87.
- Kearns, M. (1988), ‘Thoughts on hypothesis boosting’, *Unpublished manuscript* **45**, 105.
- Kim, K.-j. (2003), ‘Financial time series forecasting using support vector machines’, *Neurocomputing* **55**(1-2), 307–319.
- Kotsiantis, S. & Kanellopoulos, D. (2006), ‘Discretization techniques: A recent survey’, *GESTS International Transactions on Computer Science and Engineering* **32**(1), 47–58.
- Lauer, M. & Riedmiller, M. (2000), ‘An algorithm for distributed reinforcement learning in cooperative multi-agent systems’, in ‘In Proceedings of the Seventeenth International Conference on Machine Learning’, Citeseer.
- Lee, J. W. (2001), ‘Stock price prediction using reinforcement learning’, in ‘Industrial Electronics, 2001. Proceedings. ISIE 2001. IEEE International Symposium on’, Vol. 1, IEEE, pp. 690–695.
- Luo, Y., Liu, K. & Davis, D. N. (2002), ‘A multi-agent decision support system for stock trading’, *IEEE network* **16**(1), 20–27.
- Numpy Developers (2015–), ‘Numpy library v1.13.3’. [Online; accessed 2018-02-18].
URL: <http://www.numpy.org/>
- Nygren, K. (2004), ‘Stock prediction—a neural network approach’, *Master’s Thesis, Royal Institute of Technology, KTH, Stockholm*.
- Pang, B., Lee, L. et al. (2008), ‘Opinion mining and sentiment analysis’, *Foundations and Trends® in Information Retrieval* **2**(1-2), 1–135.
- PyData Development Team (1997–), ‘Pandas library v0.18.1’. [Online; accessed 2018-02-18].
URL: <https://pandas.pydata.org/>
- Python Software Foundation (2001–), ‘Python programming language v3.6.5’.
URL: <https://docs.python.org/3/library/>
- Quandl (2018), ‘Quandl Wiki end of day data’. [Online; accessed 2018-02-18].
URL: <https://www.quandl.com/data/WIKI>

- Quandl Developers (2006–), ‘Quandl library v3.3.0’. [Online; accessed 2018-02-18].
URL: <https://www.quandl.com/tools/python>
- Schöneburg, E. (1990), ‘Stock price prediction using neural networks: A project report’, *Neurocomputing* **2**(1), 17–27.
- Sklearn Developers (2017–), ‘Sklearn library v0.19.1’. [Online; accessed 2018-02-18].
URL: <http://scikit-learn.org/stable/>
- Sun, R. & Peterson, T. (1999), ‘Multi-agent reinforcement learning: weighting and partitioning’, *Neural networks* **12**(4-5), 727–753.
- Sutton, R. S. & Barto, A. G. (1998), *Reinforcement learning: An introduction*, Vol. 1, MIT press Cambridge.
- Thomas, A. (2016), ‘The comprehensive guide to stock price calculation’. [Online; accessed 2018-06-11].
URL: <https://blog.quandl.com/guide-to-stock-price-calculation>
- Waskom, M., Botvinnik, O., O’Kane, D., Hobson, P., Lukauskas, S., Gempferline, D. C., Augspurger, T., Halchenko, Y., Cole, J. B., Warmenhoven, J., de Ruiter, J., Pye, C., Hoyer, S., Vanderplas, J., Villalba, S., Kunter, G., Quintero, E., Bachant, P., Martin, M., Meyer, K., Miles, A., Ram, Y., Yarkoni, T., Williams, M. L., Evans, C., Fitzgerald, C., Brian, Fannesbeck, C., Lee, A. & Qalieh, A. (2017), ‘mwaskom/seaborn: v0.8.0 (july 2017)’.
URL: <https://doi.org/10.5281/zenodo.824567>
- Watkins, C. J. C. H. (1989), Learning from delayed rewards, PhD thesis, King’s College, Cambridge.
- Watkins, C. J. & Dayan, P. (1992), ‘Q-learning’, *Machine learning* **8**(3-4), 279–292.
- Wiering, M. A. & Van Hasselt, H. (2008), ‘Ensemble algorithms in reinforcement learning’, *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* **38**(4), 930–936.
- Wilder, J. W. (1978), *New concepts in technical trading systems*, Trend Research.
- Wolpert, D. H. (1992), ‘Stacked generalization’, *Neural networks* **5**(2), 241–259.

- Wooldridge, M. (2009), *An introduction to multiagent systems*, John Wiley & Sons.
- Yang, H., Chan, L. & King, I. (2002), Support vector machine regression for volatile stock market prediction, *in* ‘International Conference on Intelligent Data Engineering and Automated Learning’, Springer, pp. 391–396.
- Zhang, C. & Ma, Y. (2012), *Ensemble machine learning: methods and applications*, Springer.

Appendix A

User Manual

This appendix will include all required information for running the program.

A.1 Configuration

The file “config.py” includes a number of configuration options. This is the file to consult should the user want to change the default directory location for different files (both intermediate and final).

The folder paths that can be adjusted are:

- AGENT_PATH. This is the path where the trained agents are automatically saved after every 10th iteration. Depending on the number of agents, this can be quite big, on the order of gigabytes for hundreds of agents.
- GRAPHIC_PATH. This is the path where the graphics generated by the program will be stored.
- RESULTS_PATH. This is where the output results will be stored.

The subfolders of these folders, such as specific graphics, can also be adjusted, by changing the appropriate variables.

This file also contains the stocks to be used in the experiments. Should the user want to use a different set of stocks for training/experimenting, they can be changed here.

A.2 Running

For running the program, the libraries specified in section 4.1, together with any required dependency, must be installed, depending on which parts the user

wants to run.

The following flags specify some options for the program:

- `-p` specifies that the user wants to generate plots. Requires Matplotlib and Seaborn.
- `-l` will print all required libraries for the run of all parts of the program.
- `-t` will set training to true. The agents will be trained (this is where the code will probably spend most of the time).
- `-i` is the number of iterations through the whole training data per agent before making the testing experiments. Default is 500.
- `-s` is a seed to be used for always producing the agents with the same hyperparameters in the same order (albeit still pseudorandom). Useful if we want to stop the execution at some time and resume where we left off.
- `-n` is the number of agents to be trained per market abstraction, default is 10.

Then, `main.py` can be executed with the desired flags. For example, for training (and testing) 10 agents for each abstraction group for 500 iterations each using a seed of 9, the user would run:

```
python main.py -s 9 -n 10 -i 500 -t
```

If the agents are already trained (in cache files, as specified above) the user can simply run

```
python main.py
```

to run and test the multiagent results. This will also take a considerable amount of time, but not as much as training the individual agents. This will only work if the trained agents already exist.

If the user additionally wants graphic results, they can run:

```
python main.py -p
```

Provided Matplotlib and Seaborn are installed.